Scripts de ataque — anatomia técnica

Cada script liga o furo, mostra o abuso, e liga a correção no request seguinte. O que muda entre os dois é só o interruptor — a lógica de negócio é a mesma.

9	< 30	30–69	≥ 70
scripts	approve	challenge	deny

O interruptor que produz o furo. `NAIVE_MODE=true` desliga todos os controles. O header `X-Naive-Controls: idempotency|challenge|velocity|signals` desliga um só por request, para o script exibir furo e correção sem subir dois processos — mas o header só é lido se `ALLOW_DEMO_HEADER=true` (`app/security/naive.py`).

Fora do laboratório, `NAIVE_MODE` e `ALLOW_DEMO_HEADER` precisam ser `false` : senão um atacante desliga o antifraude só mandando um header. Score = soma dos pesos das regras que dispararam, com teto 100 (`app/engine/scorer.py`); limiares em `config/rules.yml` .

Parte 3 — o motor defendendo

Com a defesa ligada do início ao fim: idempotência, webhook e card testing.

double_charge.py · Parte 3

idempotência · 1 cobrança

cobrança indevida

O QUE EXPLORA

Retry sequencial da mesma compra. Dois `POST /payments` com a mesma `Idempotency-Key` (duplo clique, timeout do cliente, reenvio do navegador) viram duas capturas.

POR QUE FUNCIONA (TÉCNICO)

Sem uma reserva atômica da chave, cada `POST` gera um `payment_id` novo, insere uma linha nova e chama o PSP. O mock do adquirente não deduplica de propósito (`psp_mock/main.py`), então a segunda chamada cobra de verdade.

ONDE O CÓDIGO PERMITE

No caminho desligado, `claim_idempotency` em `app/storage/redis_store.py` roda `time.sleep(race_window_seconds)` e depois `self.redis.set(lock_key, request_hash, ex=30)` sem `nx=True` : o segundo `SET` só sobrescreve e os dois retornam `owner=True` . O Postgres reforça o cenário: o índice `payments_idempotency_idx` é não-único (`app/storage/postgres.py`), então duas linhas com a mesma chave convivem.

O QUE FECHA (CÓDIGO + CONFIG)

`claim_idempotency` com `self.redis.set(lock_key, request_hash, nx=True, ex=30)`: só o primeiro vira dono. `finish_idempotency` grava a resposta pronta em `idempotency:{key}` com TTL 86400. A segunda chamada cai em `if claim.stored is not None: return claim.stored` (`app/api/payments.py`) e devolve o JSON guardado, sem tocar no PSP. `_read_done` compara o `request_hash`: mesma chave com corpo diferente vira 409.

NO MUNDO REAL

Produção precisa de `SET NX` no Redis e uma constraint `UNIQUE` no banco e um PSP que deduplique pela `Idempotency-Key`. Janela residual assumida: crash entre `notify_psp` e `finish_idempotency`; nesse caso a mesma chave compartilhada com o adquirente resolve.

webhook_replay.py · Parte 3

event_id único · 409

captura forjada

O QUE EXPLORA

Reenviar o webhook legítimo do PSP (bytes e assinatura corretos, capturados em `GET :8001/sent-webhooks`) para capturar o mesmo pagamento duas vezes.

POR QUE FUNCIONA (TÉCNICO)

O HMAC prova autenticidade e integridade, mas não unicidade. Uma mensagem válida continua válida a cada reenvio enquanto o timestamp estiver dentro da tolerância (300s). Sem controle de evento, cada reenvio capturaria de novo.

ONDE O CÓDIGO PERMITE

Seria permitido se a captura não consumisse o `event_id`. O ataque mostra as duas camadas que impedem isso.

O QUE FECHA (CÓDIGO + CONFIG)

Camada 1, Redis: `claim_replay` faz `self.redis.set(f"replay:{event_id}", "1", nx=True, ex=ttl)` (`app/storage/redis_store.py`); a segunda vez retorna `False` e vira 409 "evento já processado". Camada 2, Postgres: `event_id` é PK de `webhook_events` e `insert_webhook` usa `ON CONFLICT (event_id) DO NOTHING` (rowcount diferente de 1 vira 409). Se o Redis for esvaziado (`flushdb`), o PK do banco ainda segura. A reserva vem depois da autenticação (`app/api/webhooks.py`), então assinatura ruim não queima um `event_id` legítimo.

NO MUNDO REAL

Idempotência de evento = id único do emissor + armazenamento durável da marca. O TTL do Redis só acelera o caminho feliz; a garantia de unicidade mora no banco.

webhook_tamper.py · Parte 3

MAC do corpo cru · 401

captura forjada

O QUE EXPLORA

Alterar o `amount_cents` do webhook mantendo a assinatura antiga — capturar um valor diferente do autorizado.

POR QUE FUNCIONA (TÉCNICO)

A fraude funcionaria em dois cenários: (a) o MAC ser calculado sobre um JSON re-serializado em vez dos bytes exatos; (b) o serviço não conferir o valor do webhook contra o valor autorizado, caso o segredo vaze.

ONDE O CÓDIGO PERMITE

Permitido se `webhook_is_authentic` recebesse o dicionário parseado, ou se a captura confiasse no `amount_cents` do corpo.

O QUE FECHA (CÓDIGO + CONFIG)

(1) `raw = await request.body()` lê os bytes crus antes do `json.loads`; `sign` assina `timestamp.encode() + b"." + raw_body` (`app/security/psp_signature.py`). Um dígito a mais nos bytes quebra o MAC, `_compare` fica falso e a resposta é 401. (2) Mesmo com o segredo vazado: `if int(row["amount_cents"]) != amount_cents` devolve 409 em `app/api/webhooks.py` — captura só se o valor casar com o autorizado, que não muda. (3) `_compare` confere o tamanho e usa `hmac.compare_digest` (tempo constante), e o 401 é idêntico para assinatura ruim e timestamp velho, para não dizer qual checagem falhou.

NO MUNDO REAL

Assine e verifique sempre os bytes exatos recebidos, nunca uma re-serialização (`10.0` e `10.00` são bytes diferentes). E valide a invariante de negócio — o valor autorizado — independente da assinatura.

card_testing.py · Parte 3

contadores por BIN · deny 70

cobrança indevida

O QUE EXPLORA

Validar listas de cartões roubados com micro-cobranças (R$ 1,00) no mesmo BIN. Aprovação indica cartão vivo.

POR QUE FUNCIONA (TÉCNICO)

O custo de testar é quase zero e o valor baixo passa despercebido se o motor só olhar e-mail ou IP, que o atacante troca a cada tentativa.

ONDE O CÓDIGO PERMITE

Permitido se não existissem contadores por BIN e por faixa de valor, ou se eles só contassem por e-mail/IP.

O QUE FECHA (CÓDIGO + CONFIG)

`record_velocity` mantém sorted sets deslizantes: `velocity:bin:{bin}:cards` (cartões distintos no BIN) e `velocity:ip:{ip}:low` (tentativas de valor ≤ `low_amount_cents`). `card_testing_reason` dispara (peso 70, deny) quando `distinct_cards_bin >= 4` ou `low_amount_attempts >= 5` ou a razão de negação do BIN (`bin_denies / bin_samples >= 0.6`, com pelo menos 5 amostras via `record_outcome`). Limiares em `config/rules.yml`. A razão olha só o passado — a tentativa atual entra depois, em `record_outcome`, para não diluir a própria suspeita.

NO MUNDO REAL

Contadores por BIN, por faixa de valor e por taxa de declínio, em janela deslizante. Não dependa de identificadores descartáveis (e-mail/IP). Em produção os limiares seriam bem menores que os do laboratório.

Parte 4 — furo contra correção

Cada script roda a mesma chamada com o controle desligado e depois ligado.

idempotency_race.py · Parte 4

SET NX · corrida fechada

cobrança indevida

O QUE EXPLORA

A mesma `Idempotency-Key` em N requisições simultâneas (5 threads liberadas por uma `Barrier`). É a versão concorrente do double charge.

POR QUE FUNCIONA (TÉCNICO)

Race condition de livro: um check-then-act sem atomicidade. Todas as threads leem `_read_done` = None, todas gravam, todas cobram. O `time.sleep(250ms)` do modo desligado só alarga a janela entre o GET e o SET para a barreira das threads cair dentro dela. No mundo real essa janela é o tempo real entre ler e escrever, menor, mas existe sob carga.

ONDE O CÓDIGO PERMITE

O mesmo branch desligado de `claim_idempotency`: `_read_done` (None), depois `time.sleep`, depois `set(...)` sem `nx`. Como nada serializa as threads, as cinco passam como dono e produzem cinco `payment_id` distintos.

O QUE FECHA (CÓDIGO + CONFIG)

`set(lock_key, request_hash, nx=True, ex=30)` é atômico no Redis: exatamente uma thread recebe `True`. As outras caem em `claim.waiting` e chamam `wait_idempotency` (`app/api/payments.py`), que faz polling de `_read_done` até a resposta da dona aparecer e devolvem a mesma resposta. Saldo: 1 id, 1 cobrança.

NO MUNDO REAL

Qualquer estado compartilhado sob concorrência. A correção nunca é um contador lido-e-gravado; é uma primitiva atômica: `SET NX`, `SELECT ... FOR UPDATE`, unique constraint, ou compare-and-swap.

challenge_abuse.py · Parte 4

OTP de uso único · 401/409

bypass de controle

O QUE EXPLORA

Burlar o step-up: com o controle desligado, qualquer código aprova e repetir o `verify` cobra de novo.

POR QUE FUNCIONA (TÉCNICO)

Um OTP mal implementado falha em três eixos: aceitar código errado, permitir reuso, e renovar o TTL a cada chute (deixando o atacante tentar para sempre).

ONDE O CÓDIGO PERMITE

No branch desligado de `verify_payment` (`app/api/payments.py`): não consulta o digest, chama `discard_otp`, marca `verified` e chama `notify_psp` de novo. Qualquer `code` passa e a repetição cobra de novo.

O QUE FECHA (CÓDIGO + CONFIG)

No branch ligado, `check_otp` (`app/security/otp.py`): digest é HMAC com pepper e prefixo `otp:`; comparação com `hmac.compare_digest` (tempo constante); teto de `otp_max_attempts` (3); no erro o TTL é preservado (`ex=remaining`), senão o atacante renovaria a janela a cada chute; no acerto `redis.delete(key)` garante uso único, então o replay do mesmo OTP vira 401/409. O código em claro nunca persiste — só o HMAC fica no Redis.

NO MUNDO REAL

OTP com teto de tentativas, TTL fixo, comparação em tempo constante, consumo único, e o segredo guardado apenas como hash. `EXPOSE_OTP` devolver o código no JSON é só do laboratório; em produção o OTP sai por outro canal.

velocity_shard.py · Parte 4

fingerprint global · deny 70

cobrança indevida

O QUE EXPLORA

O mesmo cartão em muitos IPs e e-mails, um par novo a cada request, para fugir de contadores por IP/e-mail.

POR QUE FUNCIONA (TÉCNICO)

Se a velocity só conta por IP e por e-mail, trocar os dois mantém cada chave em 1, sempre abaixo do limite. O cartão — o recurso que o fraudador realmente não troca de graça — some da contagem.

ONDE O CÓDIGO PERMITE

Em `app/engine/rules/velocity.py`, com `naive_velocity`, só `distinct_cards_ip` e `email_attempts` são avaliados. Em `app/engine/signals.py`, `trust_client_ip = naive_signals or naive_velocity`, então o IP de pontuação vem do JSON — trocar o campo `ip` escapa até do contador por IP.

O QUE FECHA (CÓDIGO + CONFIG)

No caminho ligado, `velocity_reason` também conta `card_attempts` (`velocity:card:{fingerprint}`) e `bin_attempts`. O fingerprint é global: a 8ª tentativa do mesmo cartão na janela vira deny peso 70, não importa IP nem e-mail. E o IP passa a ser o do socket (`_observed_ip` não lê `X-Forwarded-For`). O fingerprint é HMAC do PAN normalizado (`app/security/pan.py`), então o mesmo cartão com um espaço no meio é a mesma chave.

NO MUNDO REAL

Chaveie a velocity pela identidade que o atacante não troca sem custo (fingerprint do instrumento), não só por IP/e-mail. E conte com o IP observado no socket, não com o declarado.

forged_signals.py · Parte 4

observação contra declaração · challenge 40

bypass de controle

O QUE EXPLORA

Forjar país, IP e BIN no JSON para fugir da regra geo e do filtro de e-mail descartável.

POR QUE FUNCIONA (TÉCNICO)

Se o motor pontua com os campos que o cliente envia, o cliente controla o próprio risco: declara país/BIN coerentes e nunca cai em challenge.

ONDE O CÓDIGO PERMITE

Em `app/engine/signals.py`, no modo desligado: `scoring_ip = payload.ip`, `scoring_bin = payload.bin`, `ip_country = payload.country` (a tabela de IP nem é

consultada) e `disposable = False` (ignora o e-mail). Tudo vindo do corpo.

O QUE FECHA (CÓDIGO + CONFIG)

No modo ligado: `scoring_ip = observed_ip` (socket), `scoring_bin = payload.card[:6]` (BIN derivado do PAN real, não o declarado), `ip_country = tables.country_for_ip(scoring_ip)` e `disposable = tables.is_disposable(email)`. `geo_bin_reason` compara país do BIN contra país do IP (peso 40, challenge) e marca domínio descartável. IP privado/loopback vira `LAB_LOCAL_COUNTRY` (`app/engine/tables.py`). País desconhecido é fail-open de propósito — as outras regras seguem. `extra=forbid` nos models rejeita campo desconhecido.

NO MUNDO REAL

Separe o que o cliente afirma do que o servidor observa. BIN sai do PAN, país sai do GeoIP do IP do socket. Pontue observação, nunca declaração.

zscore_outlier.py · Parte 4 · bônus

cold start peso 0 · challenge 40

robustez estatística

O QUE EXPLORA

Z-score do valor contra o histórico do `customer_id`. O script demonstra robustez estatística: o cold start não pode gerar falso-deny e o outlier não pode se esconder.

POR QUE FUNCIONA (TÉCNICO)

Três armadilhas: aplicar peso com base pequena gera falso positivo; incluir a tentativa atual na média deixa o outlier puxar a média e se diluir; contar só capturas deixa o fraudador apagar da base o que foi barrado.

ONDE O CÓDIGO PERMITE

Seria permitido com peso aplicado em `n` pequeno, ou com a amostra sob julgamento dentro da média, ou com a base considerando apenas aprovações.

O QUE FECHA (CÓDIGO + CONFIG)

Em `app/engine/rules/zscore.py`: `if len(history) < min_history (5): return None` e `if stdev == 0: return None`; usa `statistics.pstdev` do histórico anterior, sem a tentativa atual; peso 40, no máximo challenge, nunca deny sozinho (`config/rules.yml`). `customer_amounts` (`app/storage/postgres.py`) inclui tentativas negadas na média, então o fraudador não limpa a estatística rodando o que o motor barrou.

NO MUNDO REAL

Regra estatística precisa de base mínima, exclusão da amostra sob julgamento, e nunca ser determinante sozinha. Combine com regras determinísticas.